

Sistema Integrado de Atendimento e Execução de Serviços



Tech Challenge — Fase 1

Domain-Driven Design · Clean Architecture · Spring Boot 3.4.5 · PostgreSQL 16

01

O Problema

Uma oficina mecânica de médio porte gerenciava tudo com planilhas e anotações.



Sem rastreabilidade de status

Clientes não sabiam o andamento dos serviços.
Ligações constantes e retrabalho na recepção.



Estoque descontrolado

Peças retiradas sem registro. Quantidade negativa e compras emergenciais frequentes.



Orçamentos inconsistentes

Preços alterados durante a execução.
Sem snapshot de preço, valores divergiam.

02

A Solução

MVP back-end monolítico · API RESTful · containerizado com Docker Compose

104

testes automatizados

80%+

cobertura de domínio (JaCoCo)
Validação Sonar

7

bounded contexts

8

endpoints de ciclo de vida da OS

Stack tecnológica

Java 21

Spring Boot 3.4.5

PostgreSQL 16

Flyway

JWT + Refresh Token

Docker Compose

JUnit 5 + Testcontainers

OWASP Dep. Check

Lombok

Springdoc OpenAPI

Sonar

03

Event Storming

event_storming.drawio · Domain Storytelling: domain_storytelling.drawio · 7 bounded contexts · 10 eventos · 5 políticas · 4 hot spots

7 Bounded Contexts

■ Atendimento	AG: Cliente, Veículo
■ Diagnóstico	AG: OS
■ Gestão de Serviços/Peças	AG: OS, Peça
■ Orçamento	AG: OS · SE: Notificação
■ Aprovação / Cancelamento	AG: OS, Peça
■ Execução	AG: OS
■ Entrega	AG: OS

5 Políticas (automáticas)

- OS Aberta → Iniciar Diagnóstico
- Peça adicionada à OS → Decrementar Estoque
- Estoque < mínimo → Alertar Admin
- Aprovado → Iniciar Execução
- Reprovado → Cancelar OS + Estornar Estoque

4 Hot Spots

- PA** **Mecânico disponível para diagnóstico?**
Alocação de recursos fora do escopo do MVP
- PA** **Canal de notificação? (email / SMS / push)**
ACL implementado — canal concreto em aberto
- PA** **Prazo máximo para aprovação do orçamento?**
Política de expiração fora do escopo do MVP
- PA** **Permissões por papel (Atendente/Mecânico)?**
Perfil único Admin no MVP — RBAC não implementado

Fluxos

Principal:

OS Aberta → Em Diagnóstico → Aguardando Aprovação → Em Execução → Finalizada → Entregue

Alternativo:

Orçamento Reprovado → OS Cancelada + Estorno de Estoque (atômico, mesma transação)

04

Context Map

ContextMap.drawio.pdf · padrões: Customer-Supplier · Shared Kernel · ACL · OHS

Open Host Service

- Segurança JWT é um OHS que serve todos os contextos
- Spring Security filter chain — validação HMAC-SHA384 stateless
- Todos os endpoints administrativos consomem este serviço
- Único ponto de entrada de autenticação: POST /api/auth/login

Shared Kernel

- Diagnóstico ↔ Gestão de Estoque
- Modelo Peça / EstoqueAtual compartilhado entre os dois contextos
- Mudanças no modelo exigem coordenação bilateral
- Implementado via Shared Kernel no Context Map
- Risco: acoplamento forte — mitigado por domínio estável de Peça

Customer-Supplier — cadeia principal

Atendimento

Diagnóstico

Orçamento

Aprovação / Cancelamento

Execução

Entrega

Anti-Corruption Layer

- **Aprovação/Cancelamento → Gestão de Estoque**

Estorno isolado da lógica de cancelamento do domínio de estoque

- **Orçamento → Serviço de Notificação (SE, Hot Spot PA2)**

Porta: NotificacaoService (application/port) · Stub: LogNotificacaoService

05

Design Tático

Building blocks do DDD implementados no domínio da oficina (Evans, 2003)

Value Objects

- CpfCnpj
 - Dígitos verificadores, sem UUID
- Placa
 - Antigo ABC-1234 e Mercosul ABC1D23
- StatusOS (enum)
 - validarTransicaoPara() fecha transições

Imutáveis · identificados pelo valor

Entities

- Cliente — UUID id
 - CPF/CNPJ é atributo, não identidade
- Veículo — UUID id
 - Placa é atributo de domínio
- Serviço — UUID id
- Peça — UUID id

Estado mutável · UUID persistente

Aggregate Root

- OrdemServico (único AG)
 - |— ItemServico (interno)
 - snapshot de preço do Serviço
 - |— ItemPeca (interno)
 - snapshot de preço da Peça
 - └— StatusOS (VO interno)
- Acesso somente via OrdemServicoRepository*

06

Arquitetura em Camadas

Clean Architecture · dependências apontam para o domínio · testável por camada

Infrastructure

infrastructure/

Spring Boot · JPA · Flyway · Security
REST Controllers · LogNotificacaoService (ACL stub)

Application

application/

Use Cases · Ports de entrada e saída
Orquestra o domínio sem conhecer HTTP ou JPA

Domain

domain/

Entidades · Value Objects · Aggregates
Exceções · Interfaces de Repositório
Puro Java — zero dependência de framework

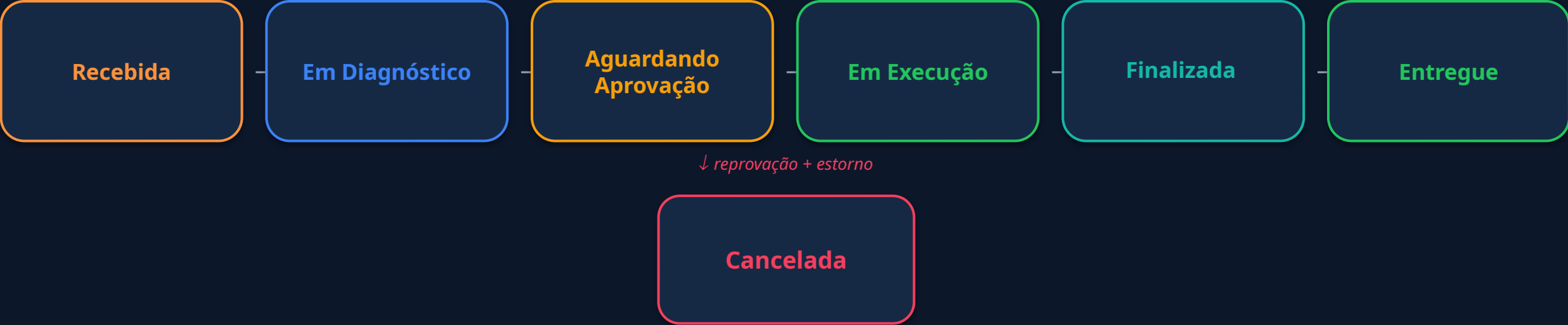
 regra de dependência

Verificável empiricamente: testes de domínio executam sem Spring Context (ms). JaCoCo enforce 80%+ em domain.* a cada build.

07

Ciclo de Vida da Ordem de Serviço

Invariante central: OS só avança — nunca retrocede. Validado em `StatusOS.validarTransicaoPara()`



Endpoints do ciclo de vida

POST /api/ordens	— Abre OS	POST .../aprovar	— Aprova → Em Execução
POST .../iniciar-diagnostico	— Inicia diagnóstico	POST .../concluir	— Conclui serviços → Finalizada
POST .../servicos + /pecas	— Adiciona itens (snapshot de preço; peça: decrementa estoque)	POST .../entregar	— Entrega → Entregue
POST .../gerar-orcamento	— Gera orçamento e notifica cliente	POST .../reprovar	— Cancela + estorna estoque → Cancelada

08

Segurança — JWT com Refresh Token

Autenticação stateless (access) + stateful (refresh) com rotação e detecção de token roubado

Access Token

Tipo: JWT · HMAC-SHA384

Validade: 15 minutos

Storage: Stateless — validado pela assinatura, sem banco

Refresh Token

Tipo: UUID opaco

Validade: 7 dias

Storage: Stateful — persiste na tabela refresh_tokens

Rotação de Refresh Tokens

Envia refresh token

Verifica no banco

Invalida token antigo

Emite novo par
access+refresh

Token já revogado?

Revoga TODOS do
usuário

`RefreshTokenService.rotacionar()` · detecção de reutilização revoga família inteira do usuário

09

Qualidade de Software

Pirâmide de testes · JaCoCo enforce 80% · OWASP Dependency Check · SonarCloud Quality Gate



104

Testes automatizados

Domain: JUnit 5 puro (sem Spring)
Application: JUnit 5 + Mockito
Integração: Testcontainers
(PostgreSQL real)



80%+

Cobertura de domínio

JaCoCo fase verify
Falha o build se < 80%
Pacote: com.oficina.mecanica.domain.*
Automatizado com Sonar na pipeline



PASS

OWASP Dep. Check

7 supressões documentadas
3 falsos positivos (DefaultServlet)
4 sem patch — risco aceito e documentado (Spring Sec + Tomcat)

10

Débitos Técnicos Conhecidos

Decisões conscientes de MVP — identificadas, documentadas, priorizadas para a próxima fase

Mappers manuais

⚠ Conversão JPA Entity ↔ Domain Object feita à mão nos adapters de persistência.

→ MapStruct — geração em compile-time, campo faltante vira erro de compilação, zero overhead em runtime.

Notificação stub

⚠ NotificacaoService é um port sem implementação concreta. E-mail e SMS não são enviados.

→ Adapter com JavaMail / SendGrid / AWS SES. Canal ainda indefinido — Hot Spot PA2 do Event Storming.

Reconstituídor com 9 parâmetros

⚠ Método fromPersistencia em OrdemServico tem 9 parâmetros (Sonar S107).

→ Encapsular em record DadosOrdemServico — 1 parâmetro, invariantes claras, sem Sonar smell.

11

Demonstração

docker-compose up --build → http://localhost:8080/swagger-ui.html

1

Login

POST /api/auth/login — obtém access token + refresh token

2

Cadastrar Cliente

POST /api/clientes — CPF validado por dígito verificador

3

Cadastrar Veículo

POST /api/veiculos — placa formato antigo ou Mercosul

4

Abrir OS

POST /api/ordens — status: Recebida

5

Fluxo completo

iniciar-diagnostico → adicionar itens → gerar-orcamento → aprovar → concluir → entregar

6

Estorno de estoque

POST .../reprovar — itens devolvidos ao estoque atômicamente, status: Cancelada

7

Consulta pública

GET /api/ordens/{id}/status — sem JWT (endpoint público)

8

Renovar Token

POST /api/auth/refresh — rotação: novo par emitido, token antigo revogado

12

O Que Foi Entregue



Back-end monolítico containerizado

Spring Boot 3.4.5 + Docker Compose · API RESTful documentada no Swagger UI



DDD — Design Estratégico

Event Storming · Context Map · Linguagem Ubíqua · 7 bounded contexts modelados



DDD — Design Tático

Value Objects · Entities · Aggregate Root (OrdemServico) · Use Cases · Repositórios



Clean Architecture

Domain / Application / Infrastructure · dependência invertida verificável pelos testes



Ciclo completo da OS

8 endpoints de ciclo de vida · snapshot de preço · estorno de estoque atômico · relatório SQL



JWT + Refresh Token

Access 15 min (HMAC-SHA384) · Refresh 7 dias com rotação e detecção de reutilização



104 testes · JaCoCo 80%+

JUnit 5 puro · Mockito · Testcontainers (PostgreSQL real) · enforce no build



OWASP Dep. Check aprovado

7 supressões documentadas · 3 falsos positivos · 4 sem patch — risco aceito e documentado



CI com SonarCloud

GitHub Actions: build → testes → JaCoCo → sonar → Quality Gate bloqueante na branch main